Python 基础知识

目录

- 一、条件判断
- if条件判断
- 二、循环语句
- 2.1 for循环
- 2.2 while循环
- 2.3退出循环

一、条件判断

- if条件判断
- 编程时经常需要检查一系列条件，并据此决定采取什么措施。在python中，if语句能让你检查程序的当前状态，并采取相应的措施。

```
if 晴天：
    出门吃火锅
if 雨天：
    在家玩游戏
```

一、条件判断

- if语句

```
if conditional_test:
    do something
```

- if-else语句

在条件测试通过时执行一个操作，在没有通过时执行另一个操作。

else有两个特性：
（1）else需与if搭配使用，它无法独立出现。必须先有if，然后才能使用else。
（2）if与else成互斥关系，else是把if判断条件无法囊括的情况全部揽到了自己这里。

```
YourAge = 20
MyAge = 5
if YourAge > MyAge:
    print("you are older than me")
if YourAge <= MyAge:
    print("you are not older than me")
else:
    print("you are not older than me")
```

if判断

优化

else判断

一、条件判断

- if-elif-else语句
- 检查超过两个的情形， 使用if-elif-else语句依次检查每个条件测试，直到遇到通过了的条件测试，通过测试后，python会跳过余下测试。

```
YourAge = 20
MyAge = 5
if YourAge > MyAge:
    print("you are older than me")
if YourAge == MyAge:
    print("we have the same age!")
if YourAge < MyAge:
    print("I am older than you")
```

if

优化

```
elif YourAge == MyAge:
    print("we have the same age!")
else:
    print("I am older than you")
```

elif

一、条件判断

- if-if语句与if-elif语句区别：

例：现有成绩单scores=[87,54,71,88,92],按照学习成绩>=90分的同学用A表示，60-89分的同学用B表示，60分以下的用C表示，重新制作一份成绩单并打印出来。

```
scores=[87,54,71,88,92]
A=[]
B=[]
C=[]
for i in scores:
    if i>=90:
        A.append(i)
    elif i>60 and i<89:
        B.append(i)
    else:
        C.append(i)
print(f'A等级:{A}')
print(f'B等级:{B}')
print(f'C等级:{C}')
```

```
A等级:[92]
B等级:[87, 71, 88]
C等级:[54]
```

```
scores=[87,54,71,88,92]
A=[]
B=[]
C=[]
for i in scores:
    if i>=90:
        A.append(i)
    if i>60 and i<89:
        B.append(i)
    else:
        C.append(i)
print(f'A等级:{A}')
print(f'B等级:{B}')
print(f'C等级:{C}')
```

```
A等级:[92]
B等级:[87, 71, 88]
C等级:[54, 92]
```

一、条件判断

- if-if语句与if-elif语句区别：

例：披萨店顾客点餐。如果顾客点了两种配料，就需要确保在其披萨中包含这些配料

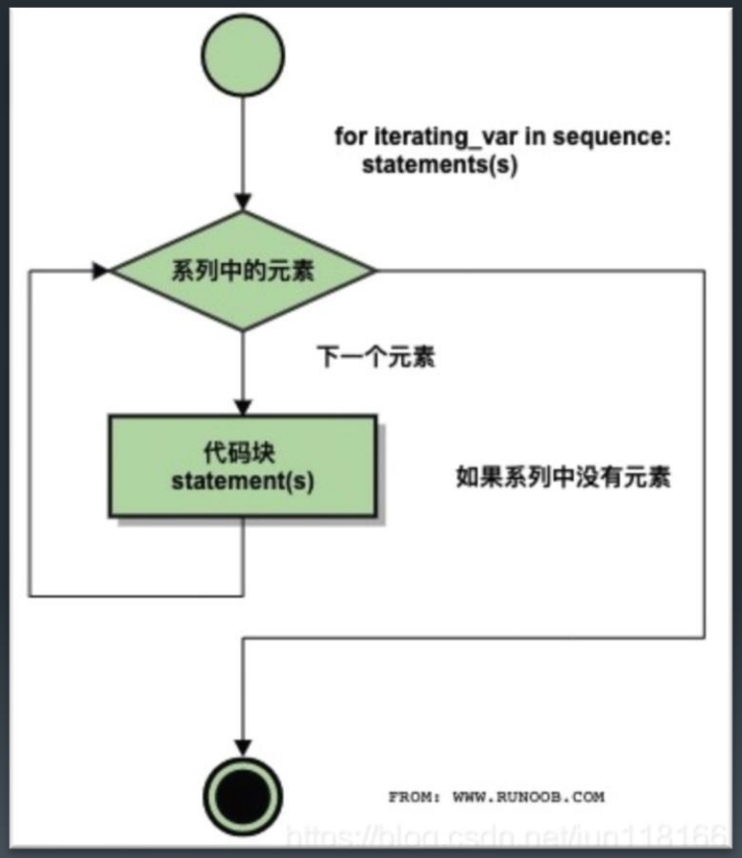
```
requested_toppings=['mushroom','extra cheese']
if 'mushroom' in requested_toppings:
    print('Adding mushrooms')
if 'extra cheese' in requested_toppings:
    print('Adding extra cheese')
```

总结：如果只想执行一个代码块，就使用if-elif-else语句；
如果要执行多个代码块就使用一系列独立的if语句

二、循环语句

- 2.1 for循环
- Python for循环可以按照从头到尾的顺序来“访问”列表里的所有元素，并且根据我们的需求对这些元素进行操作。

```
for <variable> in <sequence>:
    <statements>
else:
    <statements>
```



二、循环语句

- for循环的执行顺序：
- 使用for循环，逐个访问了NumberList中的元素，并且打印了每个元素。当我们执行完for循环以后，如果同一层级（缩进一致）还有代码未执行，则按照顺序，继续自上而下执行。

```
NumberList = [1, 2, 3, 4, 5]
for i in NumberList:
    print(i)
print(i)
```

同一层级，自上而下，顺序执行

```
1
2
3
4
5
5
```

- 程序跳出循环，继续执行和for循环处于同一层级的print()语句。

```
NumberList = [1, 2, 3, 4, 5]
for i in NumberList:
    print(i)
print(i)
```

for循环结构

循环结束以后再执行

- 本例中，这时的i在for循环结束时取值为5，所以再次打印出5。

二、循环语句

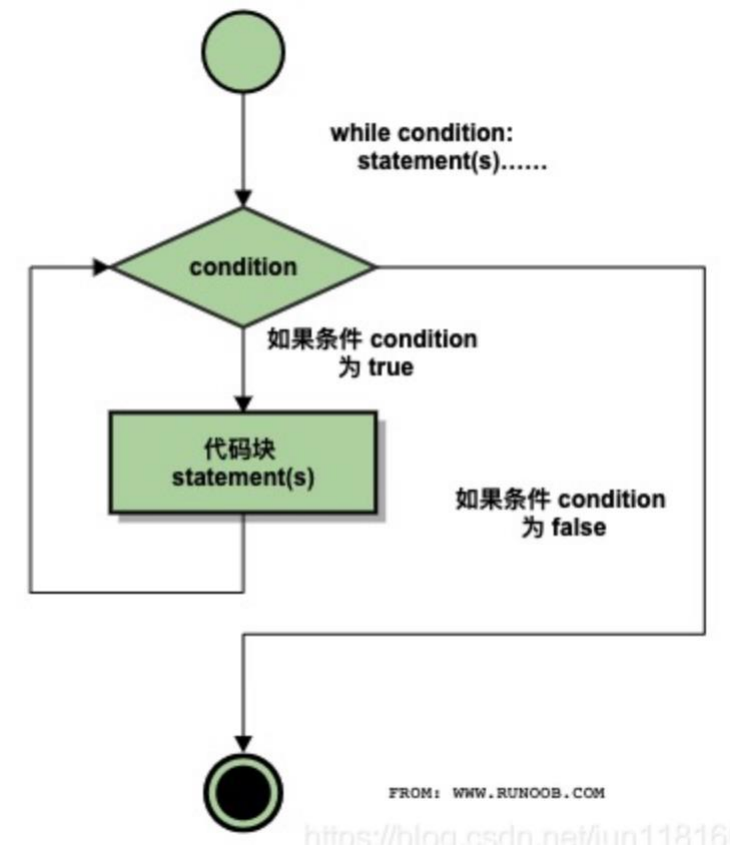
- for循环累加：
- for循环经常应用在多个数累加/累乘的计算中，也就是说需要反复多次地对大量数字进行同样地操作。

```
JDlist=[120,234.5,338,42]
sum=0
for i in JDlist:
    sum=sum+i
print(sum)
>>>734.5
```

二、循环语句

- 2.2 while循环
- Python for循环用于针对集合中的每个元素都执行一个代码块，而while循环则不断运行，直到指定的条件不满足为止。While循环也叫条件循环

```
while 判断条件(condition):
    执行语句(statements)……
```



二、循环语句

- While循环计数
- 右图中，变量j就是一个计数器，它的作用是通过自身值的不断变化来索引出列表NumberList中对应的值，推动while循环的进行。
- 在while循环里，判断条件是j<5，所以在循环语句里对变量j进行了每次加1再赋值给j的动作。随着j的数值增大，在某个时刻，判断条件j<5就会变为False，循环就会结束。

```
NumberList = [1, 2, 3, 4, 5]
j = 0
while j < 5:
    print(NumberList[j])
    j = j + 1
```

二、循环语句

- while循环与for循环区别

循环的终止条件（边界条件）不一样

- for循环的终止条件是遍历完这个list。

```
NumberList = [1, 2, 3, 4, 5]
for i in NumberList:
    print(i)
```

- 而while循环的终止条件是一个比较运算（j<5），当这个比较条件为True的时候继续执行代码，为False的时候就终止执行。

```
NumberList = [1, 2, 3, 4, 5]
j = 0
while j < 5:
    print(NumberList[j])
    j = j + 1
```

二、循环语句

- while循环累加器与计数器

```
sum= 0
counter= 0
while counter < 5:
    sum = sum + counter #累加器
    counter = counter + 1 #计数器
    print(sum)
```

- 循环过程

	第一次循环	第二次循环	第三次循环	第四次循环	第五次循环
sum	0	1	3	6	10
counter	1	2	3	4	5

```
0
1
3
6
10
```

二、循环语句

- While无限循环
- while循环是有可能一直运行的。只要判断条件为真，它就会一直执行下去。这点和for循环不一样，因为for循环是有天然的边界条件，即完成遍历。

- 比如以下代码就会永远执行，这种循环我们又叫做”死循环”。

```
Tag=True
while Tag == True:
    print('Hello tag')
```

```
Hello tag
Hello tag
Hello tag
Hello tag
Hello tag
Hello tag
Hello tag
Hello tag
Hello tag
Hello tag
Hello tag
Hello tag
```

二、循环语句

- 使用标志
- 在要求很多条件都满足才继续运行的程序中，可定义一个变量，用于判断整个程序是否处于活动状态，这个变量称为标志（flag），充当程序的交通信号灯。

```
prompt='\nTell me somethig,and I will repeat it back to you:'
prompt += "\nEnter 'quit' to end the program."

active=True  #使用标志active
while active:
    message=input(prompt)
    if message != 'quit':
        print(message)
    elif message=='quit':
        active=False
```

二、循环语句

- 2.3退出循环
- 无论是for循环还是while循环，在一个大循环里面，默认的终止条件都是边界条件。在触发边界条件之前，代码会一直循环执行。
- 如果要改变这种“默认循环执行”，可采用break语句或continue语句退出循环

```
NumberList = [1, 2, 3, 4, 5]
for i in NumberList:
    print(i)
```

小循环　大循环

- break语句

立即退出while循环，不再运行循环中余下的代码，也不管条件测试的结果如何。

- continue语句

要返回循环开头，并根据条件测试结果决定是否继续执行循环。

二、循环语句

- continue语句

continue的作用是跳过本次小循环中它后面的剩余代码，直接进入下一次迭代。

```
NumberList = [10, 4, 1, -1, 20, 6]
for i in NumberList:
    if i <= 5:
        continue
    print(i)
```

```
10
20
6
```

本例中，i依次从列表NumberList中取值进入循环，如果i大于5，判断为False，按顺序执行处于同一级的print()语句，打印出i的值；如果i小于等于5，判断为True，执行continue，结束本次小循环。直到列表NumberList中的值全部取完，整个循环就结束了。

二、循环语句

```
NumberList = [10, 4, 1, -1, 20, 6]
for i in NumberList:
    if i <= 5:
        continue
print(i)  #更改缩进
```

本例与上例的唯一不同之处在于print()语句的位置不一样。
（1）在上例中，print()包含在for循环以内并且与if判断同级，所以循环执行的次数中判断为False的有多少次，print()就要打印多少次。
（2）而在本例中，print()与for循环同级，只有当for循环结束以后，才按顺序执行print()，打印出for循环结束时最终i的值。

二、循环语句

```
NumberList = [10, 4, 1, -1, 20, 6]
for i in NumberList:
    if i <= 5:
        continue
        print(i) #再次更改缩进
```

本例与上两例的唯一不同之处也在于print()语句的位置不一样。在本例中，print()语句不仅在for循环以内并且在if判断里面，所以当i小于等于5的时候，判断为True，执行continue，程序会跳过同级的print()，结束本次小循环。而当i大于5的时候，判断为False，没有程序执行。所以整个程序没有输出。

二、循环语句

- break语句

break的作用是跳出整个大循环，终止整个循环语句的执行，让代码从循环外的第一行开始执行。

```
numberList = [10, 4, 1, -1, 20, 6, 7, -20, 2, 13]
for i in NumberList:
    if i > 10:
        print(i)
        break
```

本例中，i依次从列表 NumberList中取值，只有if判断为真，i的值大于10才会执行 print()，打印出i的值，然后执行 break结束整个循环。当i取值到20时，20>10，if判断为真，执行 print()，打印出20，然后继续执行 break，整个循环就结束了

二、循环语句

```
numberList = [10, 4, 1, -1, 20, 6, 7, -20, 2, 13]
for i in NumberList:
    if i > 10:
        print(i)
    break #更改缩进
```

本例与上一例的唯一不同之处在于break的位置不一样。
（1）在上一例中，break包含在if判断语句中，判断语句为真时，才会执行break。
（2）本例中，break与if判断语句处于同级，程序按顺序执行if判断，执行完以后再执行break。i第一次取值为10，if判断为假，程序继续执行break，整个循环结束，没有打印输出。